Copyright All rights reserved

15. 동적 메모리 할당
(Dynamic Memory Allocation)

Heesung Oh
https://github.com/3dapi

Heesung Oh
Copyright All rights reserved

- 메모리 구조
  - 코드 영역
  - 데이터 영역(Data)
    - 전역변수
    - static 변수
  - 스택(Stack)
  - 힙(Heap)

15.1 정적 할당과 동적 할당

Heesung Oh
Copyright All rights reserved

- 자동 할당 (Stack)
  - 함수 호출 시 생성 / 종료 시 자동 제거
  - 지역 변수 저장
  - 크기 제한 존재 → 스택 오버플로우 가능
- 정적 할당 (Data 영역)
  - 컴파일 시점에 크기 결정
  - 프로그램 시작 시 할당 / 종료 시까지 유지
  - 전역 변수, static 변수
  - 크기 고정 → 실행 중 변경 불가
- 동적 할당 (Heap)
  - 실행 중 필요할 때 메모리 요청 → 사용 후 자원 반환
    - 자원 요청 → 자원 확인 → 자원 사용 → 자원 반환
    - 직접 해제 필요 (free)
  - 포인터로 주소 반환

15.2 메모리 관련 함수들

Heesung Oh
Copyright All rights reserved

- 동적할당
  - 헤더 파일

    #include ⟨stdlib.h⟩
  - malloc(size_t size)
  - calloc(size_t count, size_t size)
  - realloc(void* ptr, size_t new_size)
  - 해제 : free(void* ptr)

- 메모리 조작
  - 헤더 파일

    #include ⟨string.h⟩
  - memset
  - memcpy
  - memmove

15.2.1 memset

Heesung Oh
Copyright All rights reserved

- `void* memset (void* ptr, int value, size_t num)`

- 자료형 구분 없이 메모리를 바이트 단위로 채움
  value의 하위 1바이트를 반복

```
int arr[4];
memset(arr, 0, sizeof(arr));  // 배열의 모든 바이트가 0
```

```
memset(arr, 1, sizeof(arr));  // → 0x01010101로 채워 짐
```

15.2.2 memcpy

Heesung Oh
Copyright All rights reserved

- `void* memcpy (void* dest, const void* src, size_t num)`
  - 자료형 구분 없이 지정한 바이트 수만큼 그대로 복사

```
int a = 10;
int b;
memcpy(&b, &a, sizeof(int));
```

- 주의
  - 메모리 영역이 겹치면 정의되지 않음 (UB) → memmove 사용
  - 플랫폼에 따라 결과 달라질 수 있음

```
char buffer[10] = "abcdefghi";
memcpy(buffer + 2, buffer, 5);  // 위험
```

15.2.3 memmove

Heesung Oh
Copyright All rights reserved

- `void* memmove (void* dest, const void* src, size_t num)`

- memmove
  - 메모리를 바이트 단위로 복사
  - 복사 영역이 겹쳐도 안전하게 동작

- memcpy 와 비교
  - 상대적으로 느림
  - 겹쳐도 안전

```
// 안전한 복사
char buffer[10] = "abcdefghi";
memmove(buffer + 2, buffer, 5);  // 안전
```

15.3.1 malloc 기본 사용

Heesung Oh
Copyright All rights reserved

- `void* malloc(size_t _Size)`
  - 요청한 크기만큼 힙 메모리 할당
  - 성공: 주소 반환 (void*), 실패: NULL 반환
  - malloc 메모리는 초기화되지 않음 → 필요 시 memset 등으로 초기화
  - C vs C++
    - C: void* 자동 변환 → 캐스팅 생략 가능
    - C++: 자동 변환 안됨 명시적 캐스팅 필요

```
// C++ 호환을 위한 명시적 캐스팅
int* ptr = (int*)malloc(buf_size);
```

- `void free(void* _Block)`
  - malloc / calloc / realloc으로 생성한 메모리만 해제
  - 사용 후 반드시 free 로 해제 ← 해제하지 않으면 메모리 누수

```
free(ptr_arr);
ptr_arr = NULL; // 댕글링 포인터 방지
```

15.3.2 malloc을 이용한 구조체의 동적 생성

Heesung Oh
Copyright All rights reserved

- 구조체의 동적 생성
  - 실행 중 malloc으로 메모리 확보
  - 확보한 메모리에 구조체 데이터 구성

```c
// 1. 사용자 정의 구조체.
typedef struct Player
{
  char name[32];
  int type; // 1: Knight, 2: Sorcerer
  int attack;
  int hp;
} Player;
```

```c
// 2. 구조체를 위한 동적 할당.
Player* player =
(Player*)malloc(sizeof(Player));
if(!player)
{
  printf("메모리 할당 실패\n");
  // ...
```

```c
// 3. memset으로 구조체를 0으로 초기화.
memset(player, 0, sizeof(Player));

// 4. 구조체 멤버 값 설정.
strcpy(player->name, "Knight");
player->type = 1;
player->attack = 20;
player->hp = 150;
```

```c
// 5. 동적할당 메모리 해제
free(player);

// 댕글링 포인터 방지
player = NULL;
```

15.3.3 배열 형태의 동적 메모리 할당

Heesung Oh
Copyright All rights reserved

- malloc → 연속된 메모리 확보
  - 포인터로 배열처럼 사용
  - 실행 중 크기 결정

- Ex) 문자열 복사

```
size_t len = strlen(input);

// 신규 저장공간 크기 = 문자열 길이 + 널 종료 문자('\0')
size_t buf_len = len + 1;

// 연속된 메모리를 동적으로 할당.
char* reversed = (char*)malloc(buf_len);

// 문자열 버퍼 전체를 0으로 초기화
memset(reversed, 0, buf_len);

// 입력 문자열을 뒤에서부터 읽어 역순으로 저장
for (size_t i = 0; i < len; ++i)
    reversed[i] = input[len - 1 - i];

// 동적 할당 해제
free(reversed);
```

15.3.4 구조체 배열 동적 할당 예제

Heesung Oh
Copyright All rights reserved

- malloc → 연속된 메모리 확보
- 구조체 배열처럼 사용 (items[i])
- 배열과 달리 실행 중 요소 개수(count) 설정

```c
typedef struct Item
{
  char name[40];
  int type;
  int attack;
  int defence;
} Item;

const int count = 3;
const int item_size = sizeof(Item);
const int buf_size = item_size * count;

// Item 구조체 count개에 해당하는 연속
메모리 확보
Item* items = (Item*)malloc(buf_size);

// 전체 구조체 배열을 0으로 초기화
memset(items, 0, sizeof(Item) * count);
```

```c
// 기본 능력치 설정
for (int i = 0; i < count; ++i)
{
    items[i].attack = 100;
    items[i].defence = 100;
}

// 첫 번째 아이템 설정
strcpy(items[0].name, "Sword");
items[0].type = 1;
items[0].attack = 150;


free(items);     // 동적 할당 해제
items = NULL;    // 해제 후 안전 처리
```

15.4.1 calloc: 0으로 초기화되는 할당

Heesung Oh
Copyright All rights reserved

- `void* calloc(size_t _Count, size_t _Size)`
  - _Count * _Size 바이트 할당
  - 모든 메모리 0으로 초기화
  - malloc + memset 과 동일한 효과

```c
typedef struct Enemy
{
    char name[40];
    int type;
    int attack;
} Enemy;


const int count = 3;

// 구조체 count개에 해당하는 연속 메모리 확보 + 0으로 초기화
Enemy* enemies = (Enemy*)calloc(count, sizeof(Enemy));
```

```c
// 기본 공격력 설정
for (int i = 0; i < count; ++i)
{
    enemies[i].attack = 50;
}

// 첫 번째 몬스터 설정
strcpy(enemies[0].name, "Goblin");
enemies[0].type = 1;
enemies[0].attack = 80;


free(enemies); // 동적 할당 해제
enemies = NULL;   // 해제 후 안전 처리
```

15.4.2 realloc: 크기 변경과 안전한 사용 패턴

Heesung Oh
Copyright All rights reserved

- `void* realloc(void* _Block, size_t _Size)`
  - 기존 메모리 크기 변경 → 확장 / 축소 가능
  - 새로운 메모리 할당될 수 있음
    - 기존 데이터 자동 복사
    - 실패 시 → NULL 반환 ← 기존 메모리는 그대로 유지

- 안전 패턴
  - 성공 했을 때만 교체

```
// 크기 늘리기
int* tmp = (int*)realloc(arr, sizeof(int) * new_n);
if(tmp)
{
    ptr = tmp; // 성공했을 때만 교체
```

15.5 포인터와 동적 메모리 관리

Heesung Oh
Copyright All rights reserved

- 기본 원칙
  - 할당/ 해제 한쌍
  - malloc → free 반드시 호출
- 댕글링 포인터 방지
  - 해제된 메모리를 가리키는 포인터
    → 접근 시 오류 발생 가능
  - free 이후 → 포인터를 NULL로 설정
- 소유권 (Ownership)
  - 메모리는 한 번만 해제
  - 누가 해제할지 명확히 정의
  - 반환 받은 쪽이 해제 책임
  - 생성자 역할 함수 사용

```
int* CreateIntArray(size_t n)
{
    int* arr = (int*)malloc(sizeof(int) * n);
    if(!arr)
        return NULL;
    return arr; // 소유권을 호출자에게 넘김
}
```

15.6 얕은 복사 vs 깊은 복사

Heesung Oh
Copyright All rights reserved

- 얕은 복사 (Shallow Copy)
  - 구조체 대입 → 값 그대로 복사
  - 포인터 멤버 → 주소만 복사 ← 동일한 메모리 공유
  - 문제
    - 한쪽 free → 다른 쪽 댕글링 포인터
    - 이중 해제(Double Free) 위험
- 깊은 복사 (Deep Copy)
  - 메모리 새로 할당
  - 실제 데이터까지 복사
  - 서로 독립적인 메모리

```
Player CopyPlayer(const Player* ref) {
  Player p2 = {0};    // 모든 구조체 값을 0으로 초기화.
  if (!ref || !ref->name)
    return p2;

  p2.level = ref->level;
  p2.name = (char*)malloc(strlen(ref->name) + 1);
  if (!p2.name) {
    p2.level = 0;
    return p2;
  }
  // 새로 할당한 메모리에 실제 데이터를 복사(깊은 복사)
  strcpy(p2.name, ref->name);
  return p2;
}
```

15.7.1 연속 메모리를 이용한 다차원 배열 구성

Heesung Oh
Copyright All rights reserved

- n차원 배열 → 1차원 연속 메모리 구조
  - 정적 배열과 메모리 배치가 완전히 동일

- 접근 방식 차이
  - 정적 배열 → 동적 메모리
    - arr[r][c] → arr2d[r * cols + c]
  - 인덱스 직접 계산 필요

```
int rows = 3;
int cols = 4;
int arrayCount = 4;
int* arr2d = (int*)malloc(sizeof(int) * arrayCount);

for (int r = 0; r < rows; ++r)
{
  for (int c = 0; c < cols; ++c)
  {
    arr2d[r * cols + c] = r * 10 + c;
```

```
int r = i / cols; // 행 인덱스 계산
int c = i % cols; // 열 인덱스 계산
arr2d[r * cols + c]
```

15.7.2 비 연속 메모리를 이용한 다차원 배열 구성

Heesung Oh
Copyright All rights reserved

- 분산 메모리를 다차원 배열처럼 사용
  - ppp[i] [j] [k]
  - 각 차원을 개별적으로 malloc
  - 전체 메모리는 하나로 이어져 있지 않음
- 주의
  - memcpy로 한 번에 복사 ❌
    - 연속 메모리 아님
    - 요소를 순차적으로 접근해서 복사
  - 메모리 해제
    - 연속 메모리 아님
    - 반드시 역순으로 free

15.7.2 비 연속 메모리를 이용한 다차원 배열 구성

Heesung Oh
Copyright All rights reserved

- Ex)

```
int iWidth  = 10;
int iHeight = 5;
int iDepth  = 2;

int ***ppp = NULL;

ppp = (int***)malloc(sizeof(int**) * iWidth);
for (int i = 0; i < iWidth; ++i)
{
  ppp[i] = (int**)malloc(sizeof(int*) * iHeight);
  for (int j = 0; j < iHeight; ++j)
  {
    ppp[i][j] = (int*)malloc(sizeof(int) * iDepth);
    for (int k = 0; k < iDepth; ++k)
      ppp[i][j][k] = k + j * iDepth + i * iDepth * iHeight;
    }
  }

  for (int i = 0; i < iWidth; ++i)
  {
    for (int j = 0; j < iHeight; ++j)
    {
      for (int k = 0; k < iDepth; ++k)
        printf("array[%d][%d][%d] = %d\n", i, j, k, ppp[i][j][k]);
    }
  }
```

```
for (int i = 0; i < iWidth; ++i)
{
  for (int j = 0; j < iHeight; ++j)
    free(ppp[i][j]);

  free(ppp[i]);
}

free(ppp);
ppp = NULL;
```

15.8 동적 메모리 관리 원칙

Heesung Oh
Copyright All rights reserved

- 기본 규칙
  - 할당 ↔ 해제 한 쌍 malloc / calloc / realloc → free
- 소유권
  - 누가 free할지 명확히 → 반환 받은 쪽이 해제
- 안전한 사용
  - 포인터 잃어버리지 않기: realloc → 임시 포인터 사용
  - 이중 해제 금지: 같은 주소 두 번 free ❌
  - 댕글링 포인터 방지: free 이후 → NULL 설정
- 복사 주의
  - 포인터 멤버 → 얕은 복사
  - 필요 시 깊은 복사

학습 정리

Heesung Oh
Copyright All rights reserved

- **동적 메모리**
  - 실행 중 메모리 크기 결정
  - malloc / calloc / realloc
    → 반드시 free
- **안전 규칙**
  - free 이후 → NULL 설정
    ← 댕글링 포인터 방지
  - 이중 해제 ❌
- **소유권**
  - 메모리는 한 번만 해제
    → 반환 받은 쪽이 책임
- **복사**
  - 포인터 멤버 → 얕은 복사
    → 주소만 복사
  - 필요 시 → 깊은 복사
- **메모리 함수**
  - memcpy → 겹치면 ❌
  - memmove → 겹침 OK
    → 바이트 단위 동작
- **다차원 배열**
  - 연속 메모리 → 배열과 동일
  - 비 연속 메모리 → 포인터 계층

실습 및 과제

Heesung Oh
Copyright All rights reserved

- 플레이어 접속
- 탄환 발사
- 아이템 배낭
- 문자열 분리 (SplitString)